

Data Cleaning

Tijdens het voorbereiden van de dataset voor de analyse, stuitte ik op een aantal ontbrekende waarden. Om deze dataset bruikbaar te maken, heb ik de volgende stappen ondernomen.

1. Interpolatie

Ik heb ervoor gekozen om de gaten in de temperatuurbedata te vullen met lineaire interpolatie. Het klimaat verandert geleidelijk. Als ik de temperatuur van 1850 en 1852 weet, is een schatting voor 1851 op basis van die twee jaren veel logischer dan een gemiddelde van de hele eeuw te nemen. Door te interpoleren, blijven de natuurlijke schommelingen en trends in data beter zichtbaar.

2. Opschonen per land (groupby)

In de code heb ik de data gegroepeerd per land zodat ik de lege cellen kon invullen. Je kunt de ontbrekende temperatuur in Canada niet invullen met gegevens uit Curacao. Door de functie `groupby('country')` te gebruiken, zorg ik ervoor dat gaten in de data van een specifiek land alleen wordt opgevuld met waarden uit hetzelfde land. Dit voorkomt dat de data onbetrouwbaar is.

3. Verwijderen van "ruis"

Sommige rijen hadden geen landnaam, of een ongeldige datum (zoals `0000-00-00`). Deze rijen heb ik verwijderd, zonder te weten over welk land het gaat of uit welk jaar de meting komt, voegt de data niets toe aan het onderzoek en vervuilt het de uiteindelijke grafieken.

4. Bronbestanden behouden

De opgeschoonde data heb ik opgeslagen in nieuwe tabellen in SQL (met de naam `cleaned_`)

Import & Database verbinding

```
import pandas as pd
import sqlalchemy as sa
import numpy as np

# Verbinding maken met de lokale XAMPP MariaDB database
engine = sa.create_engine("mysql+pymysql://root:@localhost/climate_watch")
print("Verbinding met database geslaagd.")
```

Cleaning

- Ik heb vijf verschillende tabellen met elk een andere structuur. In plaats van vijf keer dezelfde code te schrijven heb ik een dictionary gemaakt. Hierin vertel ik Python: als je de

stad-tabel pakt, moet je rekening houden met de kolommen `country` en `city` om de juiste locatie te bepalen.

- In de eerste regel van de `for`-loop haal ik de data op uit SQL en pas ik direct de eisen van de opdracht toe. Ik gebruik `pd.to_datetime` met `errors='coerce'` om van de datumtekst een echt tijdsobject te maken. De 'ruis' verander ik hiermee in een leeg veld, zodat ik die rijen daarna met `dropna()` makkelijk kan verwijderen
- (`sort values`) Dit is belangrijk voor de interpolatie. Je kunt alleen een lijn trekken tussen twee punten als ze op de juiste volgorde staan. Ik sorteer de data eerst op locatie en daarna chronologisch op datum.
- `groupby + transform`. Ik gebruik `groupby` om de dataset op te splitsen in kleine groepjes. Binnen dat groepje pas ik `interpolate` toe. Mocht er dan aan het begin of eind van de reeks nog iets missen, gebruik ik de fallback `fillna(x.mean())` Dit vult de laatste gaten met het gemiddelde van dat specifieke land.

```
# Configuratie per tabel: welke kolommen bepalen de unieke locatie?
tables_config = {
    'global_temperatures': [],
    'global_temp_country': ['country'],
    'global_temp_state': ['country', 'state'],
    'global_temp_major_city': ['country', 'city'],
    'global_temp_city': ['country', 'city']
}

for tabel, groepering in tables_config.items():
    # Inladen en datum filteren
    df = pd.read_sql(f"SELECT * FROM {tabel}", engine)
    df['dt'] = pd.to_datetime(df['dt'], errors='coerce')
    df_subset = df[df['dt'].dt.year >= 1980].dropna(subset=['dt']).copy()

    # Sorteren voor tijdreeks-analyse
    sort_cols = groepering + ['dt'] if groepering else ['dt']
    df_subset = df_subset.sort_values(by=sort_cols)

    # Numerieke kolommen identificeren
    target_cols = [c for c in df_subset.select_dtypes(include=[
        'number']).columns if c not in ['id', 'latitude', 'longitude']]

    # Interpolatie toepassen (per groep indien van toepassing)
    if groepering:
        for col in target_cols:
            df_subset[col] = df_subset.groupby(groepering)[col].transform(
                lambda x: x.interpolate(method='linear',
                    limit_direction='both'))
```

```

    )
    # Fallback: vul overgebleven gaten met het groepsgemiddelde
    df_subset[col] = df_subset.groupby(groepering)
[col].transform(lambda x: x.fillna(x.mean()))
    else:
        df_subset[target_cols] =
df_subset[target_cols].interpolate(method='linear', limit_direction='both')

    # Opslaan naar SQL
    df_subset.to_sql(f"cleaned_{tabel}", engine, if_exists='replace',
index=False)
    print(f"Tabel {tabel} succesvol verwerkt en opgeslagen.")

```

4 Opdrachten

1 Exploration

```

# Inladen van de opgeschoonde wereldwijde data
df_global_clean = pd.read_sql("SELECT * FROM cleaned_global_temperatures",
engine)

# Gebruik van describe() voor de algemene statistieken (Opdracht 4.1)
# Focus op de belangrijkste temperatuur-kolommen
stats = df_global_clean[['LandAverageTemperature', 'LandMaxTemperature',
'LandMinTemperature', 'LandAndOceanAverageTemperature']].describe()

# Toevoegen van de Mediaan (median) omdat describe() deze niet standaard
toont (behalve als 50%)
median = df_global_clean[['LandAverageTemperature', 'LandMaxTemperature',
'LandMinTemperature', 'LandAndOceanAverageTemperature']].median()
stats.loc['median'] = median

display(stats)

```

	LandAverageTemperature	LandMaxTemperature	LandMinTemperature	LandAndOceanAverageTemperature
count	432.000000	432.000000	432.000000	432.000000
mean	9.259451	14.964331	3.644563	15.655507
std	4.138503	4.252582	4.012910	1.229815
min	2.558000	8.090000	-2.853000	13.566000
25%	5.217000	10.710500	-0.296500	14.443500
50%	9.480500	15.207500	3.762000	15.732000
75%	13.224000	19.076250	7.500500	16.814250
max	15.482000	21.320000	9.715000	17.611000
median	9.480500	15.207500	3.762000	15.732000

Distributie van temperatuurwaarden over de jaren & trend in global average land temperatures over de tijd.

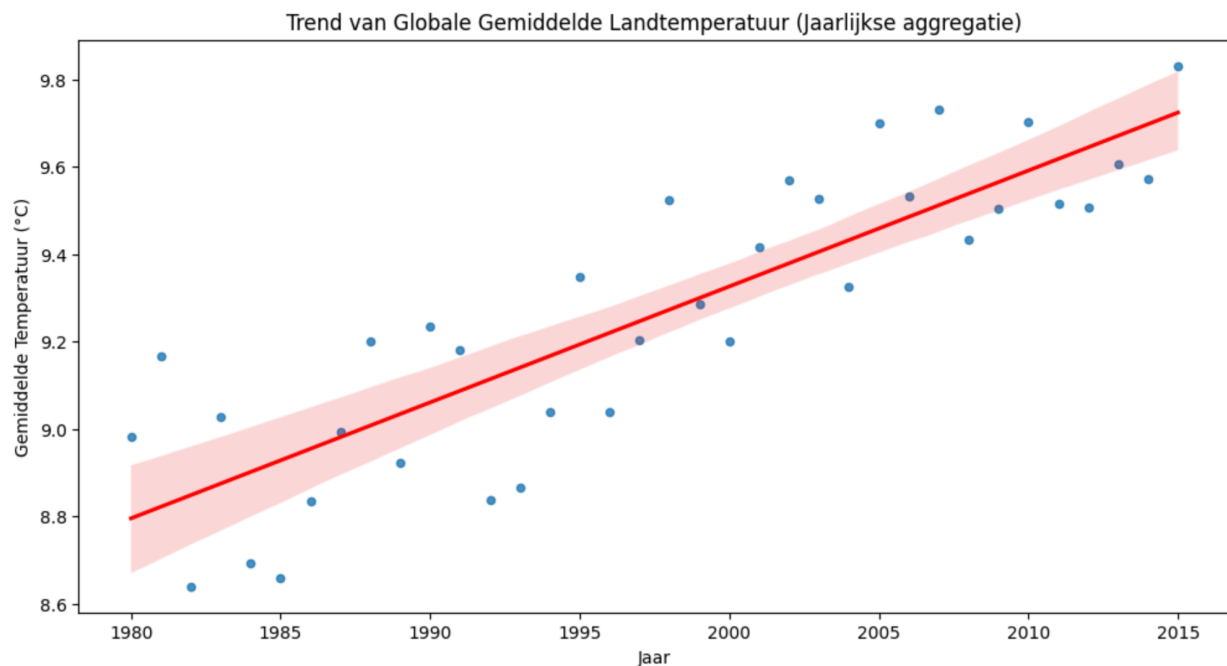
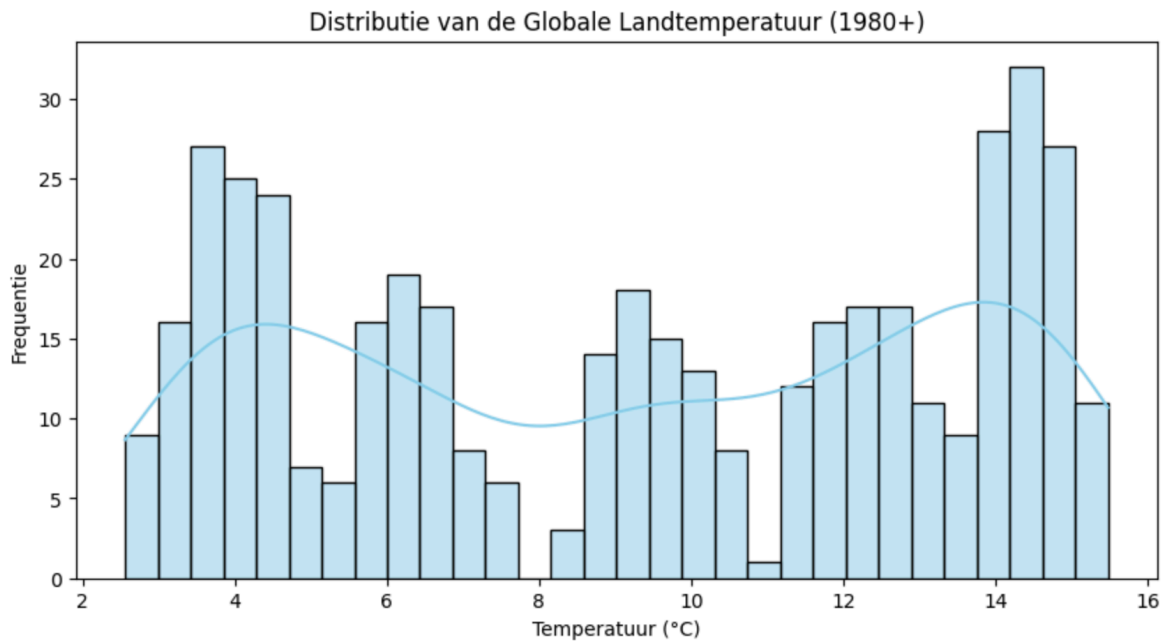
```
import matplotlib.pyplot as plt
import seaborn as sns

# Zorg dat 'dt' een datetime object is
df_global_clean['dt'] = pd.to_datetime(df_global_clean['dt'])

# 1. Distributie van de temperatuur (Histogram)
plt.figure(figsize=(10, 5))
sns.histplot(df_global_clean['LandAverageTemperature'], bins=30, kde=True,
color='skyblue')
plt.title('Distributie van de Globale Landtemperatuur (1980+)')
plt.xlabel('Temperatuur (°C)')
plt.ylabel('Frequentie')
plt.show()

# 2. Jaarlijkse groepering (Opdracht 4.2)
# Groepering per jaar om de seizoensinvloeden te middelen
df_yearly = df_global_clean.groupby(df_global_clean['dt'].dt.year)
['LandAverageTemperature'].mean().reset_index()

# 3. Visualisatie van de trend (Opdracht 4.3)
plt.figure(figsize=(12, 6))
sns.regplot(data=df_yearly, x='dt', y='LandAverageTemperature', scatter_kws=
{'s':20}, line_kws={'color':'red'})
plt.title('Trend van Globale Gemiddelde Landtemperatuur (Jaarlijkse
aggregatie)')
plt.xlabel('Jaar')
plt.ylabel('Gemiddelde Temperatuur (°C)')
plt.show()
```



2 Gerichte dataset

Tijdens de validatie van de brondata bleek dat de tabel `global_temp_country` slechts 271 rijen bevatte en dat alle datum waarden waren opgeslagen als `0000-00-00`. Dit wees op een foutieve interpretatie van het datum formaat bij het importeren in SQL.

Terwijl ik bovenstaande ging oplossen ondervond ik een `FileNotFoundError`. Na inspectie van de directory-structuur bleek dat de Jupyter kernel werd uitgevoerd vanuit een parent-directory, waardoor de relatieve paden naar de CSV-bestanden niet langer klopten. Dit heb ik gecorrigeerd door de projectstructuur te stroomlijnen: de bronbestanden (.csv) en de actieve notebooks (.ipynb) zijn in dezelfde werkmap geplaatst.

Nadat ik het .csv bestand succesvol had geïmporteerd, had ik een gerichte cleaning uitgevoerd. Uit de initiële inspectie bleek dat de dataset start in het jaar 1750, wat veel ontbrekende waarden (NULL) bevatte in de vroegere jaren.

```
import pandas as pd
import sqlalchemy as sa
import matplotlib.pyplot as plt
import seaborn as sns

engine = sa.create_engine("mysql+pymysql://root:@localhost/climate_watch")

print("Data ophalen uit MySQL...")
df_raw = pd.read_sql("SELECT * FROM global_temp_country", engine)

# Datum omzetten en filteren op 1980+
df_raw['dt'] = pd.to_datetime(df_raw['dt'])
df_clean = df_raw[df_raw['dt'].dt.year >= 1980].copy()

col_name = 'AverageTemperature'
df_clean[col_name] = df_clean.groupby('Country')[col_name].transform(
    lambda x: x.interpolate(method='linear',
    limit_direction='both').fillna(x.mean())
)

df_clean.to_sql('cleaned_global_temp_country', engine, if_exists='replace',
index=False)
```

Voor de betrouwbaarheid van de analyse heb ik de dataset beperkt tot de periode vanaf 1980. Middels lineaire interpolatie per land heb ik de resterende gaten in de temperatuurdata gedicht. De resulterende visualisatie van de top 10 warmste en koudste landen bevestigt de integriteit van de data: er zijn logische extremen te zien zoals Canada en Rusland aan de koude kant, en de tropische landen aan de warme kant.

```
# 4. Analyse: top 10 warmste en koudste landen
country_means = df_clean.groupby('Country')[col_name].mean().sort_values()

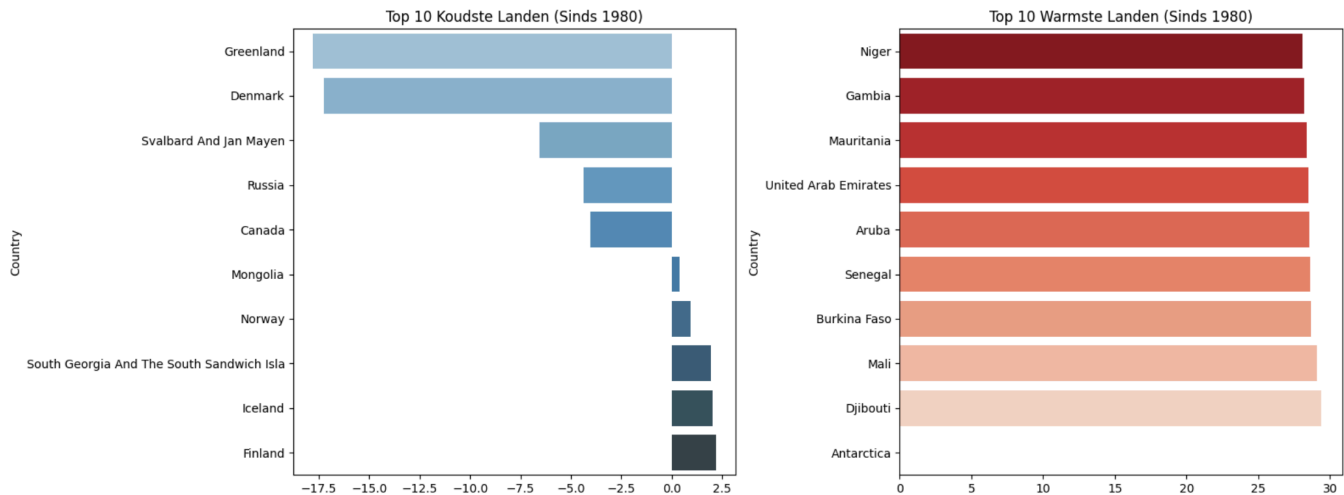
coldest = country_means.head(10)
hottest = country_means.tail(10)

# 5. Visualisatie
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
```

```
sns.barplot(x=coldest.values, y=coldest.index, ax=axes[0],
palette="Blues_d")
axes[0].set_title('Top 10 Koudste Landen (Sinds 1980)')

sns.barplot(x=hottest.values, y=hottest.index, ax=axes[1], palette="Reds_r")
axes[1].set_title('Top 10 Warmste Landen (Sinds 1980)')

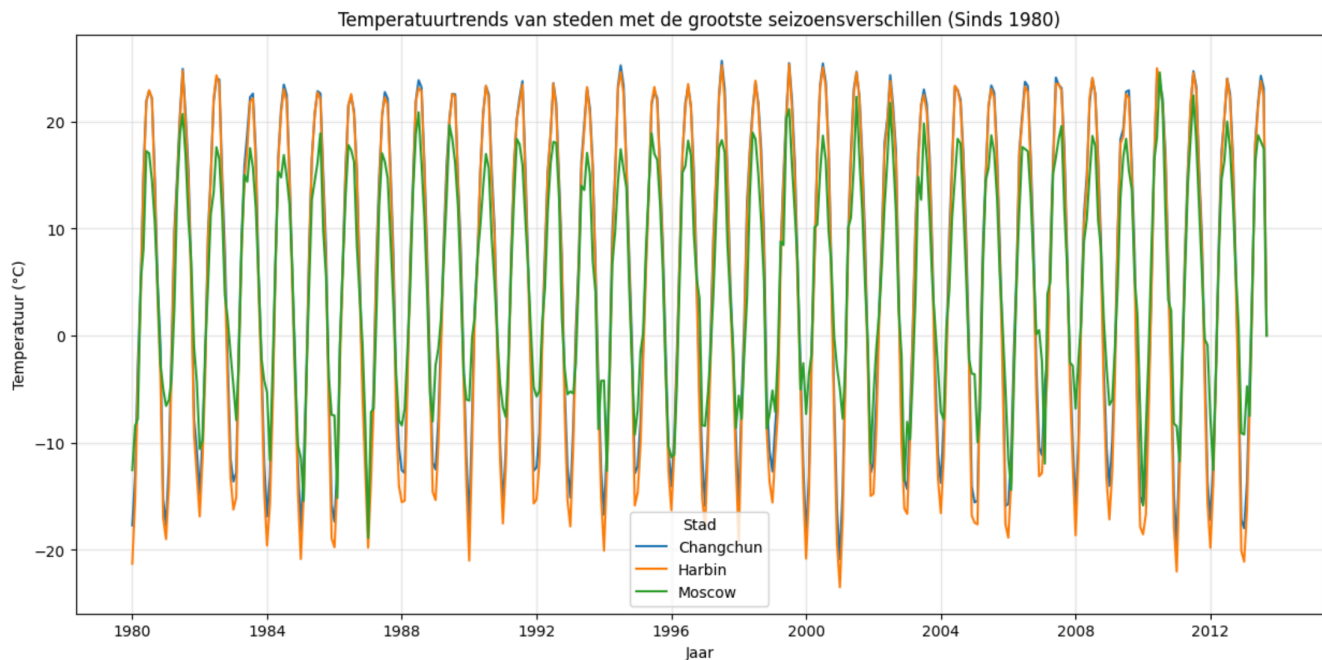
plt.tight_layout()
plt.show()
```



Bij het analyseren van de stedelijke data identificeerde ik steden met een extreem landklimaat. De stad Harbin voert de lijst aan met een temperatuur topwaarde van bijna 49 graden. Voor de visualisatie heb ik gebruikgemaakt van een tijdsreeks-analyse om de enorme jaarlijkse schommelingen in kaart te brengen.

Steden met de grootste temperatuurverschillen:

	city	max	min	diff
34	Harbin	25.357	-23.495	48.852
19	Changchun	25.671	-20.871	46.542
65	Moscow	24.580	-18.870	43.450
85	Shenyang	26.010	-14.832	40.842
79	Saint Petersburg	21.921	-18.589	40.510



3 Uncertainty Analysis

Een cruciaal aspect van klimaatdata is de onzekerheid in metingen. Uit de analyse blijkt een sterk negatief verband tussen tijd en onzekerheid. In de 18e en 19e eeuw was de onzekerheid relatief hoog (vaak boven de 1.0 graden), wat te verklaren is door minder nauwkeurige instrumenten en lagere dichtheid aan meetstations wereldwijd.

Naarmate we dichterbij het heden komen, zien we de onzekerheid dramatisch dalen tot onder de 0.1 graden. Dit valt samen met de introductie van gestandaardiseerde weerstations en later satellietmetingen.

```
import pandas as pd
import sqlalchemy as sa
import matplotlib.pyplot as plt
import seaborn as sns

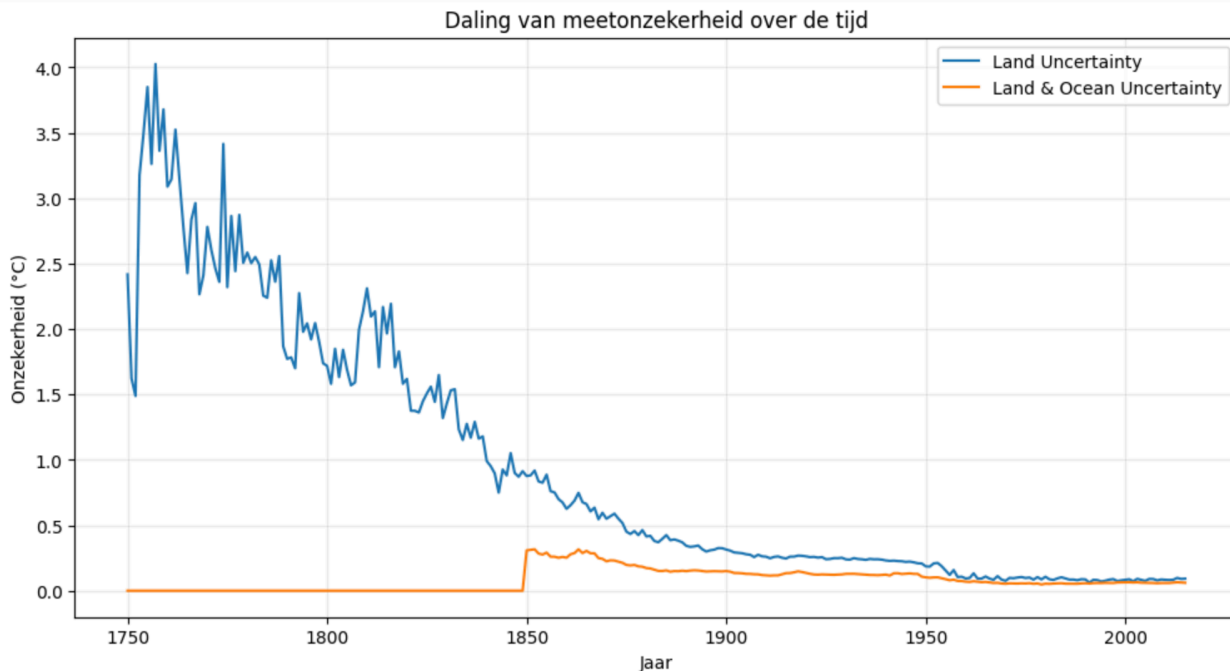
engine = sa.create_engine("mysql+pymysql://root:@localhost/climate_watch")
tabel_naam = 'global_temperatures'

try:
    query = f"""
    SELECT dt,
           LandAverageTemperatureUncertainty,
           LandAndOceanAverageTemperatureUncertainty
    FROM {tabel_naam}
    """
    df_unc = pd.read_sql(query, engine)
```

```
# 2. Data verwerken
df_unc['dt'] = pd.to_datetime(df_unc['dt'])
df_unc['year'] = df_unc['dt'].dt.year

# Groeperen per jaar voor de trend
yearly_unc = df_unc.groupby('year')
[['LandAverageTemperatureUncertainty',
'LandAndOceanAverageTemperatureUncertainty']].mean().reset_index()
```

Land vs. Oceaan: opvallend is dat de onzekerheid door de gecombineerde Land- en Oceaanmetingen vaak lager of stabiel is dan voor alleen land.



4 Geospatial Analyse

Een specifieke uitdaging in de tabel van 'major city' was het formaat van de geografische coördinaten. Deze waren opgeslagen als strings met windrichting-indicatoren (bijv. '70.05E' en '45.50N'). Voor een geospatiale analyse in Plotly moeten deze echter worden omgezet naar een numeriek 'floating point' systeem.

Ik heb een python-functie geschreven die de windrichtingen vertaalt naar het cartesische coördinatenstelsel:

- North (N) & East (E) werden omgezet naar positieve waarden.
- South (S) & West (W) werden getransformeerd naar negatieve waarden.

Door deze normalisatie konden de steden accuraat op de wereldkaart worden gepositioneerd, wat essentieel is voor het identificeren van regionale opwarmingstrends.

```

import pandas as pd
import sqlalchemy as sa
import plotly.express as px

engine = sa.create_engine("mysql+pymysql://root:@localhost/climate_watch")

# 1. Haal de data op
query = "SELECT city, latitude, longitude, average_temp FROM
cleaned_global_temp_major_city"

try:
    df_city_raw = pd.read_sql(query, engine)
    df_city_raw = df_city_raw.rename(columns={'average_temp': 'temp'})

# 2. Verbeterde functie voor coördinaten
def convert_coords(coord):
    if coord is None or str(coord).strip() == "":
        return 0.0

    coord = str(coord).upper().strip()

    # Als het S of W is, maken we het getal negatief
    if 'S' in coord or 'W' in coord:
        return -float(coord[:-1])
    # Als het N of E is, halen we alleen de letter weg
    elif 'N' in coord or 'E' in coord:
        return float(coord[:-1])

    # Voor het geval er al pure getallen in staan
    return float(coord)

# Toepassen op beide kolommen
df_city_raw['lat'] = df_city_raw['latitude'].apply(convert_coords)
df_city_raw['lon'] = df_city_raw['longitude'].apply(convert_coords)

# 3. Groeperen en Plotten
df_city_map = df_city_raw.groupby(['city', 'lat', 'lon'])
['temp'].mean().reset_index()

fig = px.scatter_geo(df_city_map,
                    lat='lat',
                    lon='lon',
                    hover_name='city',
                    color='temp',
                    color_continuous_scale=px.colors.sequential.YlOrRd,

```

```

        title='Geospatiale Analyse: Gemiddelde Temperatuur
per Wereldstad',
        projection="natural earth")

fig.show()

print("De kaart is succesvol gegenereerd!")

except Exception as e:
    print(f" Er ging iets mis: {e}")

```

Geospatiale Analyse: Gemiddelde Temperatuur per Wereldstad

